MINTIN: Maxout-Based and Input-Normalized Transformation Invariant Neural Network

Jingyang Zhang*[1], Kaige Jia*[1], Pengshuai Yang†, Fei Qiao*§, Qi Wei*, Xinjun Liu‡, Huazhong Yang*
*Beijing National Research Center for Information Science and Technology (BNRist),
Dept. of Electronic Engineering, Tsinghua University
†College of Shenyuan Honors, Beihang University
‡Dept. of Mechanical Engineering, Tsinghua University
§qiaofei@tsinghua.edu.cn

***Abstract*—Convolutional Neural Network (CNN) is a powerful model for image classification, but it is insufficient to deal with the spatial variance of the input. This paper presents a Maxout-based and input-normalized transformation invariant neural network (MINTIN), which aims at addressing the nuisance variation of images and accumulating transformation invariance. We introduce an innovative module, the Normalization, and combine it with the Maxout operator. While the former focuses on each image itself, the latter pays attention to augmented versions of input, resulting in fully-utilized information. This combination, which can be inserted into existing CNN architectures, enables the network to learn invariance to rotation and scaling. While the authors of TI-POOLING acclaimed that they reached state-of-the-art results, ours reach a maximum decrease of 0.71%, 0.23% and 0.51% in error rate on MNIST-rot-12k, half-rotated MNIST and scaling MNIST, respectively. The size of the network is also significantly reduced, leading to high computational efficiency.**

Keywords*—*transformation invariant, Normalization, Maxout.

I. Introduction

Over recent years, the landscape of computer vision has been extensively pushed forward, and advances leading to desirable results in various applications of deep learning have been achieved. These improvements largely attribute to the adoption of Deep Neural Network (DNN), especially the Convolutional Neural Network (CNN) [1].

The typical structure of a basic CNN consists of convolutional and pooling layers armed with the sufficient number of parameters. They extract local, hierarchical information with different kernels applied on small patches of the image. With these layers, we could gain rich feature representation and propagate it through the following fully-connected layers.

However, both convolutional and pooling kernels merely focus on a patch, i.e., a small part of the image (e.g., 4*4 pixels). So they only present us with internal dependencies between neighboring pixels rather than the global structural information of the whole image. This can be harmful since datasets in computer vision often contain nuisance variations which can be regarded as prior structural knowledge. E.g., when recognizing handwritten digits, noticeable rotations of the digit can confound the network since there is a great difference between the rotated one and the original one. Such variations are obviously detrimental to image classification unless being treated properly. Furthermore, appropriate treatment will lead to transformation invariance, which is a sort of generalization capacity that allows the network to adapt to different transformation and that every computer vision algorithm desires.

There are several ways to incorporate prior structural knowledge and deal with variations. The first is data augmentation[2]. It is a widely-used technique in machine learning, which enlarges the dataset by casting transformation on original samples according to our prior knowledge. TI-POOLING[3] combines data augmentation with its TI-POOLING operator, achieving better performance. Other methods, like SIFT[4], aim at extracting pre-defined features that are already transformation invariant. STN[5], however, focuses on transforming the input to its canonical appearance and therefore gets rid of variation.

While data augmentation and TI-POOLING make use of a set of representations generated from the input, SIFT and STN pay more attention to each original image. Their concerns vary from others and few works have taken them both into consideration. Inspired by STN and TI-POOLING, we propose a neural network frame (called MINTIN) that exploits not only each original image but also its transformed versions. MINTIN mainly consists of three parts. The first is a novel Normalization module which attempts to pre-calibrate the input based on prior knowledge. The following stage is the Pre-distortion operation that reflects the variations left over. The last part is the Maxout operator[6] which takes the element-wise maximum of representations corresponded with distorted images.

With this network, we achieve the error rate of 1.57% on MNIST-rot-12k[7], 1.23% on half-rotated MNIST and 0.96% on scaling MNIST. Respectively, they have a decrease of 0.71%, 0.23% and 0.51% compared with TI-POOLING, whose results have been claimed as state-of-the-art in classifying images with nuisance transformations. The following are several innovations and contributions underlying MINTIN.

The Normalization module. We originally contrive this module, which is a simple-to-implement yet effective operation without heavy extra calculation. With the assistance of pre-calibration brought by this module, the transformation set needed for the following Pre-distortion can be pretty small, leading to significantly reduced computational complexity. As illustrated in section IV, the ratio of MINTIN's computation to that of TI-POOLING under three datasets are 0.333, 0.538 and 0.556, respectively.

Cascade calibration. Unlike any conventions, we deliber-

[1]These two authors contribute equally to this work.

ately divide the acquisition of invariance into two parts, with the first in Normalization and the second in Maxout. Besides, we focus on each image itself in Normalization but then on the whole set of transformed versions in Maxout, leading to exploited information both within each individual (local dependencies) and among its group (global dependencies). Thus, by applying cascade calibration with the combination of Normalization and Maxout, we diminish variations as much as possible and achieve maximum transformation invariance.

Dynamic and flexible frame. Both the Normalization and the Maxout operator are adjustable rather than immutable. Without a noticeable decrease in accuracy, one can freely make adjustments to cope with different tasks with specific requirements on error rate and computational cost.

The rest of this paper will be organized as follow: other works related to the image classification independent of variation is illustrated in section II; the network topology and other method details are discussed in section III; we will present experimental results and relevant discussion in section IV; section V is the conclusion of our approach.

II. Related Works

There are mainly four methods concerning with transformation invariance—data augmentation, TI-POOLING, SIFT and STN. The first two aim at each single input, while the last two aim at a whole set of transformed images.

A. Data augmentation

Data augmentation[2] enlarges the dataset with transformed instances of original data according to prior knowledge. With CNN's strong expressiveness, it is able to adapt to the transformation. However, enlarged dataset indicates extensive training time and computational resources. The increase in parameters, which is necessary to cope with variation, can also pose a serious risk of over- or under-fitting.

B. TI-POOLING

TI-POOLING[3] is based on data augmentation and MIL[8]. It introduces "TI-POOLING operator" which takes the maximum of augmented versions of representations. However, the network size of TI-POOLING can be extremely large since it receives a great amount of augmentation as input, which seems not that practical in real-world tasks especially when the dataset already contains a lot of instances.

C. SIFT

The approach of SIFT[4] is to extract pre-defined features that are already transformation invariant. This kind of features, however, needs to be specifically designed each time since the situation can vary a lot under different variation, resulting in possible lack of efficiency.

D. STN

STN[5] incorporates a spatial transformer layer to transform the input to its canonical appearance. Obviously, this operation leads to extra parameters and computation, increasing the burden of the network. Besides, sometimes the output of the transformer layer is not desirable enough, i.e., is not totally canonical.

III. Method description

A. Global structure of the whole frame

MINTIN mainly consists of three parts: the Normalization module, the Pre-distortion and a multi-channel CNN with the Maxout. A typical network topology is shown in figure 1.

B. The Normalization Module

The term "normalization" indicates that this module serves to transform the input according to prior information about the variation, e.g., rotation and scaling, making images become normal or canonical.

Since it's hard and costly to transform the input to its totally unbiased position, this stage only acts as pre-calibration, which aims to reduce the randomness of the transformation, rather than to completely calibrate the input as STN does. The post-processing will be responsible for the rest variance. Therefore, in this module, we apply some simple pixel-level statistical algorithms to finish normalization without increasing much computation.

There are several advantages of the Normalization module:

- Although the variation is not completely removed, its range has been narrowed. E.g., from the example shown in figure 2, it can be seen that the original variation, a random angle from 0 to 2π, has been reduced to a regular angle within the set $\{0°, -90°, -180°, 90°\}$. In contrast to approaches without normalization, e.g., TI-POOLING, this leads to a much smaller transformation set necessary for the following procedure, and the channel number of the neural network can be significantly reduced, resulting in much less computational complexity.
- Compared with STN, our Normalization operation is much simpler and more convenient to apply. Also note that the calculation amount of simple statistical method is tiny in comparison with the training and utilization of spatial transformer layer.

C. Pre-distortion

Inspired by data augmentation and TI-POOLING, before employing the CNN, we define a set of transformations Φ, which reflects the possible present variations left in the dataset after normalization. More specifically, $\Phi = \{\phi_1, \phi_2, ..., \phi_n\}$, where $\phi_i(\cdot)$ denotes a transformation function, and $\phi_i(x)$ denotes the transformed version of image x and n specifies the number of transformations in this set (note that Φ almost always includes identity transformation). Then a collection of images will be generated according to Φ just like conventional data augmentation (which can be regarded as deliberate distortion), all of which are later sent into CNN for feature extraction.

D. Multi-channel Convolutional Neural Network and Maxout

To correspond with transformed images in an efficient manner, we construct the topology of CNN with multiple channels like [9], allowing transformed versions generated by Pre-distortion to be simultaneously propagated through series

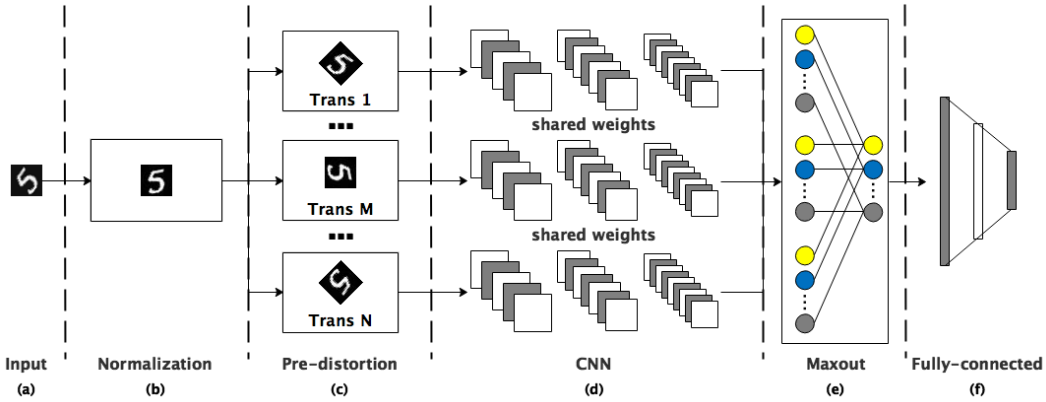

Fig. 1. (a) The input contains nuisance variations. (b) In the Normalization module, the image is calibrated but may be left with some variance. The figure only shows the best case. (c) Similar to data augmentation, we cast pre-distortion to the image at this stage, reflecting the present variance. (d) Multi-channel CNN receives augmented versions correspondingly. Shared weights enable an efficient feature extraction. (e) The Maxout operator takes the element-wise maximum of features belonging to each transformed image. (f) The output of the Maxout is sent into fully-connected layers to acquire final result.

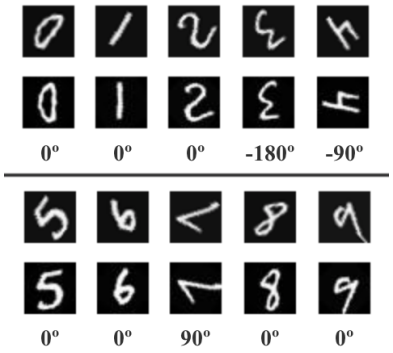

Fig. 2. The first and third rows show the images from the MNIST-rot-12k dataset, whose prior known variation is rotation with an angle randomly chosen from 0 to 2π. The second and fourth rows show the corresponding output of our Normalization module. The angle between the present appearance of the output images and their truly canonical appearance is annotated under each of them. Here positive angles are counterclockwise and vice versa. One can easily note that the original variation (from 0 to 2π) has been reduced to a set containing only four angles, $0°$, $-90°$, $-180°$ and $90°$.

of convolutional layers and pooling layers. For simplicity and clarity, we use $f_\theta(x)$ to denote extracted feature vector of input x, where θ specifies a set of parameters of all the kernels and weights. Thus the features we get are a set of $f_\theta(\phi_i(x))$, $i = 1, 2, ..., n$, each of which corresponds to a transformed image. Here n is the number of both transformations and channels since they are exactly the same.

After representations or features have been gained, conventional data augmentation will treat each of them as non-related individual. In our approach, however, we incorporate the Maxout operator[6] which takes the element-wise maximum of all $f_\theta(\phi_i(x))$.

$$g_\theta(x) = \max_{\phi_i \in \Phi} f_\theta(\phi_i(x)) \tag{1}$$

Here $g_\theta(x)$ is the final representation of image x.

According to the Lemma 1 in [10] and [8], a function defined in equation 1 is transformation invariant in many cases. Therefore, the output of the Maxout becomes as less dependent on variations as possible. Finally, we send the maximum representation to fully-connected layers, through which the final result is acquired. The back-propagation of the network is the conventional and standard one so we will not discuss it in detail here.

As the previous Normalization module has already reduced some variance, having its focus on each input (local dependencies), the Maxout operator deals with the rest variation, paying attention to the corresponding set of images (global dependencies). These two operations together lead to fully-deployed information of the data and in turn maximum invariance. Besides, both Normalization and Maxout are flexible enough to be adjusted. One can freely alter these modules for specific needs. E.g., one can apply more complicated algorithms for the Normalization or enlarge the transformation set and increase channel number if higher accuracy is expected; or inversely, if less computation is desired, one can still get satisfying results with simple algorithms and fewer channels.

IV. Experiments

MNIST[11] is a common dataset to test the performance of computer vision algorithms. Here we use three different versions of MNIST for experiments. The first two incorporate rotation variation and the third one includes scaling variation. The network topology is the same for three experiments and is shown in table I. Only the transformation sets designed for Pre-distortion are different.

A. *MNIST-rot-12k*

MNIST-rot-12k[7] is a dataset consisting of hand-written digits originally from MNIST, yet is randomly rotated with an angle from 0 to 2π. It contains 12000 training images and 50000 test images. For rotation variation, a statistical method[12] based on moment invariants is applied in the Normalization module. It helps to define the orientation of principal axis and in turn normalize the input. We totally train for 10000 iterations with batch size 64, which is approximately only 54 epochs. Dropout[13] is used to prevent over-fitting. Besides conducting MINTIN, we also reproduce the method of TI-POOLING. The results are shown in table II.

TABLE I. NETWORK TOPOLOGY

Layers&Operations	Parameters&Size
Normalization	Simple statistical algorithms
Convolution	kernel:3 × 3, stride:1, padding:1, channels:40
ReLU	
Max-pooling	kernel:2 × 2, stride:2
Convolution	kernel:3 × 3, stride:1, padding:1, channels:80
ReLU	
Max-pooling	kernel:2 × 2, stride:2
Convolution	kernel:3 × 3, stride:1, padding:1, channels:160
ReLU	
Max-pooling	kernel:2 × 2, stride:2
Linear	channels:500
ReLU	
Maxout	
Dropout	ratio:0.5
Linear	channels:10

TABLE II. RESULTS ON MNIST-ROT-12K

Method	Channels	Error,%	Ops,$\times 10^7$	Relative Ops
TI-POOLING	4	2.47	5.14	0.167
	8	1.88	10.28	0.333
	24	1.61	30.84	1
MINTIN	4	1.76	5.14	0.167
	8	1.59	10.28	0.333
	24	1.57	30.84	1

"Channels" in the table refers to the number of angles (uniformly drawn from 0 to 2π) in the transformation set when applying Pre-distortion. E.g., 4 channels indicates that the transformation set is $\Phi = \{0°, 90°, 180°, 270°\}$. "Ops" refers to the number of multiplication needed to finish a complete propagation of a single image. This figure reflects how much computational source the network will take. Here we only accurate to the second decimal place.

When using the same number of channels, our approach always achieves better results than TI-POOLING, attaining the maximum improvement of 0.71%. When reaching comparable error rate, the computation of our approach is only 0.333 of that of TI-POOLING (MINTIN with 8 channels and TI-POOLING with 24 channels). Thus, while keeping high accuracy, our method is able to reduce parameter number and lessen the size of the network.

B. Half-rotated MNIST

MNIST-rot-12k contains 12000 training images and 50000 test images, which is not typical and appropriate for a dataset. Besides, a handwritten digit rotated by more than 90° is not that common in reality. So we decide to use another version of MNIST, which is more close to real-world circumstances. It is first introduced in [5], and each of the digit is randomly rotated with angle from $-\frac{\pi}{2}$ to $\frac{\pi}{2}$. Just like original MNIST, this dataset consists of 60000 training images and 10000 test images. This time we train for 30000 iterations. The results are shown in table III.

With this dataset, our MINTIN diminishes the computation by nearly a half, since MINTIN with 7 channels performs better than TI-POOLING with 13 channels.

TABLE III. RESULTS ON HALF-ROTATED MNIST

Method	Channels	Error,%	Ops,$\times 10^7$	Relative Ops
TI-POOLING	7	1.44	9.00	0.538
	13	1.46	16.71	1
MINTIN	7	1.32	9.00	0.538
	13	1.23	16.71	1

C. Scaling MNIST

Those two experiments mentioned above only address on the rotation. To validate our approach with other variations, we artificially introduce scaling distortion to original MNIST images. More specifically, we randomly resize each digit (actually takes an area of 20×20 pixels) horizontally and vertically to $0.6 - 1.4$ times the length of the original digit with the stride being 0.1 times. Then the transformation set with 9 channels, for example, is $\Phi = \{0.6, 0.7, 0.8, 0.9, 1.0, 1.1, 1.2, 1.3, 1.4\}$. For the Normalization, we simply judge the boundary of the digit by comparing the gray scale value with threshold 0.5 and then resizing the area of the digit to 20×20 pixels. We train for merely 10000 iterations, with results presented in table IV.

TABLE IV. RESULTS ON SCALING MNIST

Method	Channels	Error,%	Ops,$\times 10^7$	Relative Ops
TI-POOLING	5	1.52	6.43	0.556
	9	1.32	11.57	1
MINTIN	5	1.01	6.43	0.556
	9	0.96	11.57	1

The results clearly show that our approach can also adapt to scaling variation. Under this situation, we achieve an average enhancement of 0.435% in accuracy. Also note that MINTIN reaches a better result with only 0.556 of total multiplications of TI-POOLING, which again proves the high computation efficiency of our network frame.

V. CONCLUSION

In this paper, we present a Maxout-based and input-normalized transformation invariant neural network (MINTIN). We introduce an innovative Normalization module to pre-calibrate the input, which is simple to implement and brings few extra computational complexity. With the combination of the Normalization and the Maxout, we can exploit the information both within each input and among its transformed versions, resulting in maximum invariance and improved performance. The Normalization module also keeps the size of transformation sets at a low level, enabling the network to function well with a rather small computational amount. Furthermore, MINTIN is flexible since neither the Normalization module or the channel number is fixed. One can make full use of MINTIN according to specific tasks or needs by applying different algorithms to the Normalization module and changing the number of channels.

ACKNOWLEDGEMENT

The authors would like to acknowledge support from National Natural Science Foundation of China under grant No.91648116 and National Key R&D Program of China under grant No.2017YFC1500601. We would also acknowledge support from Hisilicon technologies Co.,Ltd.

REFERENCES

[1] A. Krizhevsky, I. Sutskever, and G. E. Hinton, "Imagenet classification with deep convolutional neural networks," in Advances in neural information processing systems, 2012, pp. 1097–1105.

[2] D. A. Van Dyk and X.-L. Meng, "The art of data augmentation," Journal of Computational and Graphical Statistics, vol. 10, no. 1, pp. 1–50, 2001.

[3] D. Laptev, N. Savinov, J. M. Buhmann, and M. Pollefeys, "Ti-pooling: transformation-invariant pooling for feature learning in convolutional neural networks," in Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition, 2016, pp. 289–297.

[4] D. G. Lowe, "Object recognition from local scale-invariant features," in Computer vision, 1999. The proceedings of the seventh IEEE international conference on, vol. 2. Ieee, 1999, pp. 1150–1157.

[5] M. Jaderberg, K. Simonyan, A. Zisserman et al., "Spatial transformer networks," in Advances in Neural Information Processing Systems, 2015, pp. 2017–2025.

[6] I. J. Goodfellow, D. Warde-Farley, M. Mirza, A. Courville, and Y. Bengio, "Maxout networks," arXiv preprint arXiv:1302.4389, 2013.

[7] H. Larochelle, D. Erhan, A. Courville, J. Bergstra, and Y. Bengio, "An empirical evaluation of deep architectures on problems with many factors of variation," in International Conference on Machine Learning, 2007, pp. 473–480.

[8] J. Wu, Y. Yu, C. Huang, and K. Yu, "Deep multiple instance learning for image classification and auto-annotation," in Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition, 2015, pp. 3460–3469.

[9] J. Bromley, I. Guyon, Y. LeCun, E. Säckinger, and R. Shah, "Signature verification using a" siamese" time delay neural network," in Advances in Neural Information Processing Systems, 1994, pp. 737–744.

[10] D. Laptev and J. M. Buhmann, "Transformation-invariant convolutional jungles," in Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition, 2015, pp. 3043–3051.

[11] Y. LeCun, C. Cortes, and C. J. Burges, "Mnist handwritten digit database," AT&T Labs [Online]. Available: http://yann. lecun. com/exdb/mnist, vol. 2, 2010.

[12] M.-K. Hu, "Visual pattern recognition by moment invariants," IRE transactions on information theory, vol. 8, no. 2, pp. 179–187, 1962.

[13] N. Srivastava, G. E. Hinton, A. Krizhevsky, I. Sutskever, and R. Salakhutdinov, "Dropout: a simple way to prevent neural networks from overfitting." Journal of machine learning research, vol. 15, no. 1, pp. 1929–1958, 2014.